GUIA COMPLETO — EXPLICAÇÃO DOS COMANDOS CSS

Dashboard de controle de despesas | Material para explicação em sala de aula

Objetivo: explicar o que cada seletor, propriedade, valor e pseudo-classe faz no CSS apresentado. O foco é permitir que o professor percorra o arquivo linha por linha e demonstre o efeito de cada regra.

Antes de começar: HTML define a estrutura dos elementos; CSS controla sua aparência e organização; JavaScript normalmente implementa ações e regras de negócio, como adicionar ou excluir uma despesa.

1. Como ler uma regra CSS

```
.seletor {
  propriedade: valor;
}
```

• Seletor: indica quais elementos HTML receberão a regra. Exemplo: body, #app, .card.

• Propriedade: indica o aspecto que será alterado. Exemplo: color, padding, display.

• Valor: informa como a propriedade será aplicada. Exemplo: #fff, 16px, flex.

• Chaves { }: delimitam o bloco de declarações.

• Dois-pontos : separam a propriedade do valor.

• Ponto e vírgula ;: encerra uma declaração CSS.

• Ponto (.) no seletor: seleciona uma classe HTML. Jogo da velha (#): seleciona um ID.

Exemplo HTML: `<div id="app" class="painel">` pode ser selecionado por `#app` ou `.painel`.

2. Variáveis CSS e :root

```
:root {
  --fundo: #121212;
  --fundo-card: #1c1c1c;
  --fundo-card-hover: #232323;
  --borda: #2e2e2e;
  --texto: #f5f5f5;
  --texto-suave: #a3a3a3;
  --accent: #aa3bff;
  --accent-hover: #8f2ee0;
  --accent-suave: rgba(170, 59, 255, 0.15);
  --erro: #ff6b6b;
  --erro-fundo: rgba(255, 107, 107, 0.08);
  --erro-borda: rgba(255, 107, 107, 0.35);
  --radius: 12px;
}
```

• :root: pseudo-classe que seleciona o elemento raiz do documento (normalmente html). Declarar variáveis aqui permite reutilizá-las em diferentes partes da página.

• --nome-da-variavel: sintaxe de uma propriedade personalizada CSS. O nome começa obrigatoriamente com dois hífens.

• var(--fundo): função que recupera o valor guardado na variável.

• #121212, #f5f5f5 etc.: cores em hexadecimal; cada par representa intensidade de vermelho, verde e azul.

• rgba(170, 59, 255, 0.15): cor com canais vermelho, verde e azul, mais transparência (alfa). Alfa 0 é transparente; 1 é opaco.

• 12px: medida em pixels CSS; neste caso será usada para arredondar cantos.

Por que isso importa? Uma alteração em --accent atualiza todos os elementos que usam var(--accent), mantendo o padrão visual do sistema.

3. Seletor universal e regras do body

```
* {
```

```
  box-sizing: border-box;
}

body {
  margin: 0;
  background: var(--fundo);
  color: var(--texto);
  font-family: system-ui, "Segoe UI", Roboto, sans-serif;
  font-size: 16px;
  line-height: 1.45;
}
```

• *: seletor universal; seleciona todos os elementos.

• box-sizing: border-box: faz a largura e altura declaradas incluírem conteúdo, padding e borda. Facilita o cálculo do tamanho dos componentes.

• body: seleciona o corpo do documento HTML.

• margin: 0: remove a margem externa padrão aplicada pelo navegador ao body.

• background: define o fundo do elemento. Aqui recebe a variável --fundo.

• color: define a cor do texto. Não é a cor de fundo.

• font-family: define a família tipográfica. O navegador tenta system-ui, depois Segoe UI, Roboto e por fim uma fonte genérica sem serifa.

• font-size: 16px: define o tamanho-base do texto.

• line-height: 1.45: define a altura da linha como 1,45 vez o tamanho da fonte. Ajuda na legibilidade.

Diferença fundamental: margin é espaço do lado de fora da caixa; padding é espaço entre o conteúdo e a borda.

4. Área principal: #app e barra lateral

```
#app {
  display: flex;
  min-height: 100vh;
}

#sidebar {
  position: sticky;
  top: 0;
  width: 340px;
  flex-shrink: 0;
  background: #181818;
  border-right: 1px solid var(--borda);
  padding: 32px 28px;
}
```

• #app / #sidebar: seletores de ID; correspondem a elementos com id="app" e id="sidebar" no HTML.

• display: flex: ativa o Flexbox. Os filhos diretos são organizados em linha por padrão.

• min-height: 100vh: altura mínima de 100vh. A unidade vh corresponde à altura da área visível; 100vh equivale a 100% dela.

• position: sticky: mantém o elemento aderido a uma posição durante a rolagem, dentro dos limites do contêiner.

• top: 0: define a distância de zero em relação ao topo quando o sticky é ativado.

• width: 340px: define largura de 340 pixels CSS.

• flex-shrink: 0: impede que o item flexível encolha para caber no espaço disponível.

• background: #181818: define a cor de fundo da barra lateral.

• border-right: cria borda apenas no lado direito. 1px é espessura, solid é estilo contínuo e var(--borda) é a cor.

• padding: 32px 28px: define 32px em cima/baixo e 28px à esquerda/direita.

Quando a barra lateral e o conteúdo são filhos de #app, o Flexbox os coloca lado a lado. A barra ocupa 340px e o conteúdo pode ocupar o restante.

5. Textos, títulos e separadores da sidebar

```
.sidebar__titulo {
  margin: 0 0 24px;
```

```
  font-size: 14px;
  font-weight: 600;
  text-transform: uppercase;
  letter-spacing: 1px;
  color: var(--texto-suave);
}

.sidebar__rotulo-total {
  margin: 0;
  font-size: 14px;
  color: var(--texto-suave);
}

.sidebar__valor-total {
  margin: 4px 0 0;
  font-size: 34px;
  font-weight: 700;
  color: var(--texto);
}

.sidebar__divisor {
  border: none;
  border-top: 1px solid var(--borda);
  margin: 24px 0;
}
```

• .classe: seleciona todos os elementos que possuem aquela classe.

• margin: 0 0 24px: shorthand com três valores: topo 0, lados 0 e parte inferior 24px.

• font-size: tamanho da fonte.

• font-weight: 600 / 700: peso da fonte; números maiores costumam representar texto mais grosso. A disponibilidade depende da fonte.

• text-transform: uppercase: apresenta letras em maiúsculas visualmente, sem necessariamente alterar o texto original.

• letter-spacing: 1px: adiciona espaço entre caracteres.

• border: none: remove todas as bordas.

• border-top: define somente a borda superior.

No separador, a borda padrão é removida e uma linha superior de 1px é criada. Isso é comum ao estilizar elementos hr.

6. Lista de categorias e alinhamento

```
.sidebar__lista-categorias {
  list-style: none;
  margin: 0;
  padding: 0;
}

.resumo__item {
  display: flex;
  justify-content: space-between;
  align-items: center;
  gap: 12px;
  padding: 10px 0;
  border-bottom: 1px solid var(--borda);
}

.resumo__item:last-child {
  border-bottom: none;
}
```

• list-style: none: remove marcadores de listas, como as bolinhas de ul.

• justify-content: space-between: distribui os itens no eixo principal, deixando espaço entre eles e encostando-os nas extremidades.

• align-items: center: centraliza os itens no eixo transversal.

• gap: 12px: cria 12px de espaço entre os filhos do contêiner flex ou grid.

• padding: 10px 0: 10px em cima/baixo e zero nas laterais.

• border-bottom: adiciona uma linha inferior.

• :last-child: pseudo-classe que seleciona o elemento que é o último filho do pai.

A combinação permite mostrar o nome da categoria de um lado e o valor do outro. O último item não recebe a linha inferior.

7. Conteúdo principal e Flexbox em coluna

```
#conteudo {
  flex: 1;
  padding: 32px;
  display: flex;
  flex-direction: column;
  gap: 24px;
}
```

• flex: 1: forma abreviada de propriedades flexíveis; faz o item crescer para ocupar o espaço livre no contêiner flex.

• padding: 32px: aplica 32px de espaço interno nos quatro lados.

• flex-direction: column: organiza os filhos verticalmente, um abaixo do outro.

• gap: 24px: espaço entre as seções internas do conteúdo.

O conteúdo principal usa Flexbox em coluna: cabeçalho, formulário, lista e demais seções podem ficar empilhados.

8. Grid: quatro cards no cabeçalho

```
.cabecalho {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  gap: 16px;
}
```

• display: grid: ativa CSS Grid no elemento.

• grid-template-columns: define a estrutura e a quantidade de colunas.

• repeat(4, 1fr): repete quatro vezes uma coluna de 1fr.

• 1fr: uma fração do espaço disponível no grid; as quatro colunas ficam com a mesma largura.

• gap: 16px: cria espaço entre linhas e colunas quando aplicável.

Resultado esperado em tela larga: quatro cards alinhados em uma linha. O número de colunas será reduzido nas media queries.

9. Cards de serviço e sombras

```
.servico {
  display: flex;
  flex-direction: column;
  gap: 12px;
  background: var(--fundo-card);
  border: 1px solid var(--borda);
  border-radius: var(--radius);
  padding: 20px;
  box-shadow: 0 4px 16px rgba(0, 0, 0, 0.35);
}
```

• display: flex: organiza os filhos com Flexbox.

• flex-direction: column: empilha título, texto e botão.

• background: pinta o fundo do card.

• border: 1px solid ...: borda contínua de um pixel.

• border-radius: arredonda os cantos; o valor vem da variável --radius.

• padding: 20px: espaço interno de 20px em todos os lados.

• box-shadow: cria sombra. Os valores são deslocamento horizontal (0), deslocamento vertical (4px), desfoque (16px) e cor/transparência (rgba...).

A sombra ajuda a separar visualmente o card do fundo, dando sensação de profundidade.

10. Títulos, textos e flex: 1

```
.servico__titulo {
  margin: 0;
  font-size: 18px;
  font-weight: 600;
  color: var(--texto);
}

.servico__texto {
  flex: 1;
  margin: 0;
  font-size: 14px;
  color: var(--texto-suave);
}
```

• margin: 0: remove a margem padrão de elementos como h2 e p.

• font-size: 18px / 14px: define tamanhos diferentes para título e descrição.

• font-weight: 600: deixa o título seminegrito, conforme a fonte.

• flex: 1 no texto: permite que a área de descrição cresça e ocupe o espaço livre do card. Isso pode ajudar a alinhar os botões na parte inferior quando os cards têm a mesma altura.

• color: define a cor do texto.

11. Botões que são links e efeito hover

```
.servico__botao {
  display: flex;
  justify-content: center;
  align-items: center;
  gap: 8px;
  margin-top: 8px;
  padding: 10px 16px;
  border: 1px solid var(--accent);
  border-radius: 999px;
  font-size: 15px;
  font-weight: 600;
  color: #c084fc;
  text-decoration: none;
  transition: background 0.2s ease;
}

.servico__botao:hover {
  background: var(--accent-suave);
}
```

• justify-content: center: centraliza os filhos no eixo principal.

• align-items: center: centraliza os filhos no eixo transversal.

• margin-top: 8px: cria espaço externo acima do botão.

• padding: 10px 16px: 10px vertical e 16px horizontal.

• border-radius: 999px: valor alto que deixa o botão em formato de cápsula.

• text-decoration: none: remove decoração padrão do texto do link, como sublinhado.

• transition: background 0.2s ease: anima mudanças no background durante 0,2 segundo, usando curva ease.

• :hover: pseudo-classe aplicada enquanto o ponteiro está sobre o elemento.

O seletor .servico__botao:hover é mais específico que .servico__botao e altera o fundo quando ocorre hover.

12. Cartões e título de cartão

```
.card {
  background: var(--fundo-card);
  border: 1px solid var(--borda);
  border-radius: var(--radius);
  padding: 24px;
  box-shadow: 0 4px 16px rgba(0, 0, 0, 0.35);
}

.card__titulo {
  margin: 0 0 20px;
  font-size: 18px;
  font-weight: 600;
  color: var(--texto);
}
```

• As propriedades de .card repetem o padrão visual do card de serviço: fundo, borda, cantos arredondados, espaço interno e sombra.

• padding: 24px: neste card, o espaço interno é maior.

• margin: 0 0 20px: remove margens superior e laterais e deixa 20px abaixo do título.

Reutilizar padrões de card mantém consistência visual entre as áreas da aplicação.

13. Formulário: Flexbox, campos e seletores combinados

```
.form-nova-despesa {
  display: flex;
  flex-direction: column;
  gap: 16px;
}

.campo {
  display: flex;
  flex-direction: column;
  gap: 6px;
}

.campo label {
  font-size: 14px;
  font-weight: 500;
  color: var(--texto-suave);
}

.campo input,
.campo select {
  width: 100%;
  background: var(--fundo);
  border: 1px solid var(--borda);
  border-radius: 8px;
  padding: 12px 14px;
  color: var(--texto);
  font-size: 16px;
  font-family: inherit;
}
```

• .campo label: seleciona elementos label que estejam dentro de um elemento com classe campo.

• .campo input, .campo select: a vírgula separa dois seletores; as mesmas declarações são aplicadas aos inputs e selects descendentes de .campo.

• width: 100%: tenta ocupar toda a largura disponível do elemento pai.

• border-radius: 8px: arredonda os cantos do campo.

• font-family: inherit: herda a família tipográfica do elemento pai.

• font-size: 16px: define o tamanho da fonte dentro do campo.

A estrutura em coluna coloca o rótulo acima do campo. O gap define a distância entre eles.

14. Foco do campo: :focus e outline

```
.campo input:focus,
.campo select:focus {
  outline: 2px solid var(--accent);
  outline-offset: 1px;
  border-color: var(--accent);
}
```

• :focus: seleciona o campo que está com foco, seja por clique ou navegação pelo teclado.

• outline: desenha um contorno externo ao elemento; diferente de border, não ocupa espaço no layout.

• 2px solid: espessura de 2px e linha contínua.

• outline-offset: 1px: afasta o contorno um pixel da borda do campo.

• border-color: altera somente a cor da borda.

Esse destaque melhora a orientação visual e a acessibilidade, mostrando qual campo está ativo.

15. Botão de adicionar e estado hover

```
.botao-adicionar {
  background: var(--accent);
  color: #fff;
  border: none;
  border-radius: 8px;
  padding: 12px 16px;
  font-size: 16px;
  font-weight: 600;
  font-family: inherit;
  cursor: pointer;
  transition: background 0.2s ease;
}

.botao-adicionar:hover {
  background: var(--accent-hover);
}
```

• color: #fff: texto branco.

• border: none: remove a borda padrão.

• cursor: pointer: mostra o cursor de mão, indicando que o elemento é interativo.

• transition: suaviza a mudança de cor.

• :hover: troca o fundo para a variável --accent-hover enquanto o mouse está sobre o botão.

A aparência do botão não executa o cadastro. Para cadastrar uma despesa, é necessária uma ação no HTML/JavaScript ou no back-end.

16. Mensagem de erro

```
.mensagem-erro {
  margin: 16px 0 0;
  padding: 12px 14px;
  background: var(--erro-fundo);
  border: 1px solid var(--erro-borda);
  border-radius: 8px;
  color: var(--erro);
  font-size: 14px;
}
```

• margin: 16px 0 0: 16px acima, zero nas laterais e zero abaixo.

• padding: 12px 14px: espaço interno vertical e horizontal.

• background: fundo vermelho suave e translúcido.

• border: contorno com cor de erro.

• color: texto na cor de erro.

O CSS define como a mensagem aparece. A lógica da aplicação decide em que situação ela deve ser exibida.

17. Lista de despesas e itens

```
.lista-despesas {
  list-style: none;
  margin: 0;
  padding: 0;
  display: flex;
  flex-direction: column;
  gap: 10px;
}

.despesa {
  display: flex;
  justify-content: space-between;
  align-items: center;
  gap: 12px;
  background: var(--fundo-card-hover);
  border: 1px solid var(--borda);
  border-radius: 12px;
  padding: 16px 20px;
}
```

• list-style: none: remove os marcadores padrão de uma lista.

• margin: 0 / padding: 0: remove os espaços padrão do navegador na lista.

• flex-direction: column: empilha as despesas verticalmente.

• gap: 10px: distância entre um item de despesa e outro.

• justify-content: space-between: separa informações e ações nas extremidades do item.

• background: var(--fundo-card-hover): aplica o tom de fundo definido para o card em estado alternativo/hover.

• border-radius: 12px: arredonda os cantos do item.

• padding: 16px 20px: 16px vertical e 20px horizontal.

18. Informações, ações e valor da despesa

```
.despesa__info {
  display: flex;
  flex-direction: column;
  gap: 6px;
}

.despesa__titulo {
  margin: 0;
  font-size: 16px;
  font-weight: 600;
  color: var(--texto);
}

.despesa__acoes {
  display: flex;
  align-items: center;
  gap: 16px;
}

.despesa__valor {
  margin: 0;
  font-size: 18px;
  font-weight: 700;
  color: var(--accent);
}
```

• .despesa__info: agrupa título e categoria em uma coluna.

• .despesa__titulo: estiliza o título da despesa, removendo sua margem padrão.

• .despesa__acoes: coloca valor e botão de excluir lado a lado.

• align-items: center: alinha verticalmente os elementos do grupo de ações.

• gap: 16px: distância entre valor e botão.

• .despesa__valor: destaca o valor com fonte de 18px, peso 700 e cor de destaque.

19. Botão de excluir e múltiplas transições

```
.despesa__botao-excluir {
  background: transparent;
  border: 1px solid var(--borda);
  border-radius: 8px;
  padding: 6px 12px;
  font-size: 13px;
  font-weight: 500;
  font-family: inherit;
  color: var(--texto-suave);
  cursor: pointer;
  transition: background 0.2s ease, color 0.2s ease;
}

.despesa__botao-excluir:hover {
  background: var(--erro-fundo);
  color: var(--erro);
}
```

• background: transparent: fundo transparente.

• font-weight: 500: peso intermediário da fonte.

• cursor: pointer: indica que pode ser clicado.

• transition: background ..., color ...: anima duas propriedades: fundo e cor do texto.

• .despesa__botao-excluir:hover: no hover, aplica fundo e texto na paleta de erro.

O clique de exclusão precisa ser implementado pela aplicação. O CSS só apresenta o botão e seus estados visuais.

20. Categoria e mensagem vazia

```
.despesa__categoria {
  align-self: flex-start;
  background: var(--accent-suave);
  border: 1px solid rgba(170, 59, 255, 0.35);
  border-radius: 999px;
  padding: 3px 10px;
  font-size: 12px;
  font-weight: 500;
  color: #c084fc;
}

.mensagem-vazia {
  margin: 0;
  padding: 24px;
  text-align: center;
  color: var(--texto-suave);
  border: 1px dashed var(--borda);
  border-radius: 10px;
}
```

• align-self: flex-start: permite que este item se alinhe no início do eixo transversal, independentemente do alinhamento geral dos demais itens.

• border: 1px solid rgba(...): borda com transparência.

• border-radius: 999px: aparência de etiqueta arredondada.

• padding: 3px 10px: pouco espaço vertical e mais espaço horizontal.

• text-align: center: centraliza o texto dentro da área.

• border: 1px dashed ...: borda de um pixel com traços, em vez de linha contínua.

A mensagem vazia pode ser mostrada quando não há despesas. A condição de exibição não é definida pelo CSS.

21. Responsividade: media queries

```
@media (max-width: 1200px) {
  .cabecalho {
    grid-template-columns: repeat(2, 1fr);
  }
}

@media (max-width: 768px) {
  .cabecalho {
    grid-template-columns: 1fr;
  }

  #app {
    flex-direction: column;
  }

  #sidebar {
    width: 100%;
    min-height: auto;
    border-right: none;
    border-bottom: 1px solid var(--borda);
  }

  #conteudo {
    padding: 20px;
  }
}
```

• @media: regra condicional que aplica CSS quando uma condição de mídia é verdadeira.

• max-width: 1200px: as regras internas valem quando a largura da viewport é de até 1200px.

• repeat(2, 1fr): reduz o grid para duas colunas iguais.

• max-width: 768px: aplica as regras em telas de até 768px.

• grid-template-columns: 1fr: cria uma única coluna que ocupa o espaço disponível.

• flex-direction: column: no celular, sidebar e conteúdo ficam um abaixo do outro.

• width: 100%: a barra lateral ocupa a largura disponível.

• min-height: auto: deixa a altura mínima seguir o conteúdo, em vez de impor um valor específico.

• border-right: none: remove a borda direita.

• border-bottom: cria uma separação horizontal abaixo da barra lateral.

• padding: 20px: reduz o espaço interno do conteúdo em telas menores.

Viewport é a área visível da página no navegador. Os breakpoints de 1200px e 768px foram escolhas deste projeto; não são valores obrigatórios para todos os sites.

22. Atenção: corrigir caracteres da cópia

Na mensagem original, alguns símbolos parecem ter sido escapados pela formatação. Se esses caracteres estiverem literalmente no arquivo .css, devem ser corrigidos.

```
/* Correto: comentário CSS */
* {
  box-sizing: border-box;
}

/* Correto: seletor de ID */
#app {
  display: flex;
}

/* Correto: transição */
.servico__botao {
  transition: background 0.2s ease;
}
```

• Não use barra invertida antes do asterisco do seletor universal: o correto é *.

• Não use barra invertida antes de #app: o correto é #app.

• Comentários CSS usam /* texto */.

• Em transition, escreva o nome da propriedade sem asteriscos: background, não *background*.

Se os caracteres estranhos apareceram apenas ao colar a mensagem no chat, mas não estão no arquivo original, não é necessário alterar o CSS.

23. Proposta de demonstração prática

Teste ao vivo	Alteração	Conceito demonstrado
1. Tema	Mude --accent para outra cor hexadecimal.	Variáveis e reutilização.
2. Espaçamento	Altere padding: 20px para 32px em .servico.	Espaçamento interno.
3. Grid	Troque repeat(4, 1fr) por repeat(2, 1fr).	Colunas do CSS Grid.
4. Alinhamento	Remova temporariamente justify-content: space-between.	Distribuição no Flexbox.
5. Foco	Clique em um campo do formulário.	Pseudo-classe :focus.
6. Hover	Passe o mouse no botão.	Pseudo-classe :hover e transition.
7. Celular	Reduza a janela para menos de 768px.	Media query e responsividade.

24. Perguntas para verificar a aprendizagem

• Qual é a diferença entre seletor de classe e seletor de ID?

• O que :root representa e por que declarar variáveis nele?

• Qual é a diferença entre margin e padding?

• O que muda quando usamos display: flex?

• Para que serve flex-direction: column?

• Qual é a função de grid-template-columns e do valor 1fr?

• O que significa :hover? E :focus?

• Qual a diferença entre border e outline?

• O que a propriedade transition faz?

• O que uma media query permite fazer?

• O CSS é responsável por salvar uma despesa no banco de dados?

25. Resumo final: o que o aluno precisa levar

Organização: seletores identificam elementos e propriedades definem sua aparência.

Identidade visual: cores, fontes, bordas, sombras e variáveis criam consistência.

Layout: Flexbox alinha elementos em uma direção; Grid organiza linhas e colunas.

Interação visual: hover e focus mostram estados do componente; transition suaviza mudanças.

Responsividade: media queries adaptam o layout a diferentes larguras.

Limite do CSS: CSS não cadastra, calcula nem persiste despesas. Ele cuida da apresentação. A lógica é responsabilidade do JavaScript e/ou do back-end.